

Option Informatique MPSI, DS n°1

Les questions de code devront être traitées en langage OCaml. Les noms de variables doivent être suffisamment clairs, et le code compréhensible.

Si vous définissez une fonction auxiliaire, précisez le type des arguments et l'objectif de la fonction.

Les questions difficiles sont notées d'une étoile *. N'hésitez pas à les sauter.

I) Autour des polynômes

Dans cette partie, on va étudier une représentation des polynômes d'entiers en OCaml.

On prend un polynôme P :

$$P(X) = \sum_{k=0}^n a_k X^k \quad \text{pour } a_0, \dots, a_n \in \mathbb{N} \quad (1)$$

On représente les polynômes en OCaml par la liste contenant $a_0, \dots, a_n$. On se donne donc le type suivant :

```
type polynome = int list;;
```

Par exemple, le polynôme $X^3 + 1$ sera représenté par la valeur OCaml suivante :

```
let cube = [1; 0; 0; 1];;
```

Q1) Écrire une fonction `degres: polynomes -> int` qui renvoie le degré d'un polynôme (c'est-à-dire n).

```
let degres p = (List.length p) - 1;;
```

```
let rec degres p = match p with  
| [] -> -1  
| _::q -> 1 + degres q;;
```

Q2) Écrire une fonction `dominant: polynomes -> int` qui renvoie le coefficient dominant d'un polynôme (c'est-à-dire a_n).

```
let rec dominant p = match p with  
| [] -> 0  
| [x] -> x  
| _::q -> dominant q;;
```

Q3) Écrire une fonction `xn: int -> polynome` qui prend en argument un entier n et renvoie le polynôme X^n

```
let rec xn n = match n with  
| 0 -> 1  
| _ -> 0::(xn (n - 1));;
```

Q4) Écrire une fonction `mul_scalaire: polynomes -> int -> polynome` qui multiplie un polynôme par un entier.

```
let rec mul_scalaire p k = match p with  
| [] -> []  
| x::q -> (k * x)::(mul_scalaire q k);;
```

Q5) Écrire une fonction `add: polynomes -> polynome -> polynome` qui réalise la somme de deux polynômes.

```
let rec add p1 p2 -> match (p1, p2) with
| [], reste | reste, [] -> reste
|x1::q1, x2::q2 -> (x1 + x2)::(add q1 q2);;

let rec eval (p: polynome) (x: int): int = match p with
| [] -> 0
| a::q -> a + x * (eval q x);;
```

Q6) Que fait la fonction `eval` ? Quelle est sa complexité ?

La fonction `eval` évalue un polynôme en un point. Elle réalise une addition et une multiplication pour chaque coefficient du polynôme, ce qui donne une complexité en $O(n)$.

Q7) * Écrire une fonction `mul: polynomes -> polynome -> polynome` qui multiplie deux polynômes.

Relation de récurrence :

$$\left(\sum_{k=0}^n a_k X^k\right) \left(\sum_{k=0}^m b_k X^k\right) = a_0 \left(\sum_{k=0}^m b_k X^k\right) + X \left(\sum_{k=0}^{n-1} a_{k+1} X^k\right) \left(\sum_{k=0}^m b_k X^k\right)$$

```
let rec mul p1 p2 -> match p1 with
| [] -> 0
|x::q -> add (mul_scalaire p2 x) (0::(mul q p2));;
```

II) Arbres binaires de recherche

II.1) Généralités

On étudie des arbres binaires de recherche, définis par induction de la manière suivante :

Définition 1 [Arbre binaire de recherche]

Un arbre binaire de recherche est :

- Soit une feuille F ne stockant rien
- Soit un noeud interne N stockant un entier x , et deux sous arbres G et D , tels que :
 - G et D sont des arbres binaires de recherche
 - G stocke uniquement des éléments plus petits que x , et D stocke uniquement des éléments plus grands que x (strictement).

On utilise le type OCaml suivant pour stocker les arbres binaires de recherche :

```
type abr =
| F
| N of abr * int * abr;;
```

Q8) Écrire une fonction `hauteur: abr -> int` qui calcule la hauteur d'un arbre binaire de recherche.

```
let rec hauteur a = match a with
| F -> -1
| N (g, _, d) -> max (hauteur g) (hauteur d);;
```

Q9) Écrire une fonction `min: abr -> int` qui renvoie la valeur minimum stockée.

```
let rec min a -> match a with
| F -> failwith "arbre vide"
| N (F, x, _) -> x
| N (g, _, d) -> min g;;
```

Q10.) Écrire une fonction recherche: `abr -> int -> bool` qui prend en argument un arbre et un entier, et renvoie un booléen indiquant si l'entier est présent dans l'arbre. Quelle est sa complexité en fonction de h la hauteur de l'arbre ?

```
let rec recherche a x -> match a with
| F -> false
| N (g, y, d) -> if x = y then true else
    if x < y then recherche g x
    else recherche d x;;
```

Q11.) Expliquer comment on peut procéder pour insérer un élément dans l'arbre, et avec quelle complexité en fonction de h .

On peut procéder sur le même principe, mais, lorsqu'on arrive sur une feuille, on la remplace par `N(F, x, F)`.

Q12.) De même pour supprimer un élément.

- si l'élément voulu est contenu dans un noeud de la forme `N(F, x, F)`, on peut simplement le remplacer par `F`.
- si le noeud contenant l'élément a un enfant égal à `F`, on peut le remplacer par le deuxième enfant. (nb : cela inclus le cas précédent)
- si le noeud a deux sous-arbres non vides, on prend le minimum de l'arbre de droite (ou le max de celui de gauche), on le supprime du sous arbre, et on le met à la place de `x`. Cela permet de garder la propriété. A noter que le min / max ne peut pas avoir deux sous-arbres non vides, donc on est sûr de ne pas répéter l'opération.

Q13.) Soit un arbre de taille n . Indiquer le meilleur et le pire cas pour la complexité des opérations en fonction de n , par rapport à la forme de l'arbre.

Meilleur cas : arbre équilibré Pire cas : Peigne

II.2) Arbres Bicolores

On va introduire une méthode pour garder les arbres équilibrés, et donc toujours avoir une bonne complexité pour nos opérations. Habituellement on utilise des couleurs, mais pour des raisons de photocopie en noir et blanc, on utilisera des formes ici.

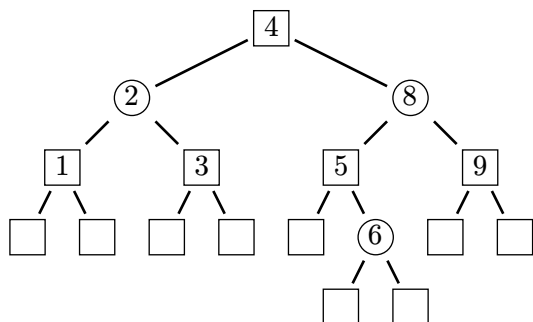
Définition 2 [Arbre bicolore]

Un arbre bicolore est un arbre binaire de recherche dans lequel on attribue une forme (carré ou rond) à chaque noeud, avec les contraintes suivantes :

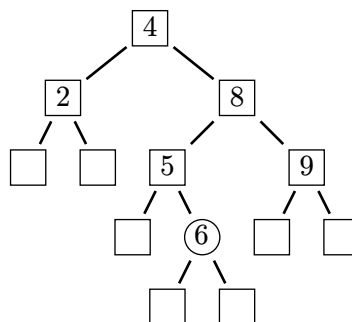
- La racine est **carré**
- Les enfants d'un noeud **rond** sont **carré**
- Tous les chemins de la racine à une feuille ont le même nombre de noeuds **carré**
- Les feuilles sont **carré**

Q14.) Les arbres suivants sont-ils des arbres bicolores ?

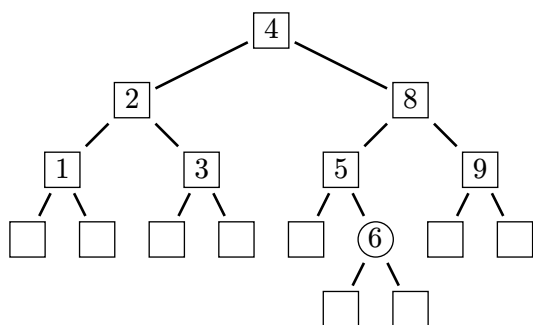
(a)



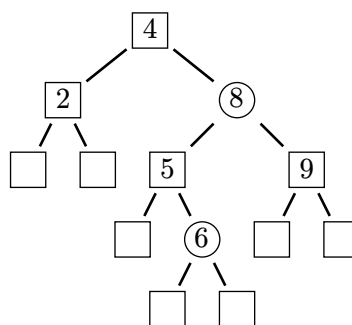
(b)



(c)



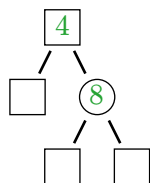
(d)



1. oui
2. non
3. oui
4. oui

Q15.) Quelle est la hauteur maximale possible pour un arbre bicolore de taille 5 ? Et de taille 17 ?

5 :



$h = 1$

17 (13 en fait) : 3

À noter que l'énoncé de cette question était un peu raté, et que il n'aidait pas vraiment à résoudre la question suivante.

Q16.) * Généralisez avec un arbre de taille $2^n + 1$, et démontrez votre résultat.

N'importe quelle majoration correcte convenait.

Pour avoir un arbre de hauteur h , il faut au moins $\lceil \frac{h+1}{2} \rceil$, et donc au moins $\frac{h}{2}$ carrés dans chaque chemin : en effet, il s'agit d'une alternance de carré et de ronds qui commence et termine par un carré, avec en tout $h + 1$ symboles.

Le plus petit arbre bicolore avec des chemins de p carrés est de taille $2^p - 1$. (Il s'agit de l'arbre complet avec uniquement des carrés).

Cet arbre ne contient que des carrés. Or, on sait que notre arbre contient au moins un rond, ce qui ajoute au moins 2 noeuds. Donc, un arbre de hauteur h est au moins de taille $2^{\frac{h}{2}} + 1$.

Donc un arbre de taille $2^n + 1$ a pour hauteur maximale $2n$.

Q17) En déduire une borne sur la hauteur d'un arbre bicolore en fonction de sa taille.

Vous pouviez admettre une majoration de la question précédente (même si elle était fausse), le raisonnement comptait.

On en déduit que, dans tous les cas, pour un arbre de hauteur h et de taille t , $t \geq 2^{\frac{h}{2}-1}$, ou $h \leq 2 \log_2(t) + 2$

Q18) En supposant que l'on puisse maintenir la propriété bicolore lors des opérations de recherche, ajout et suppression, sans coût supplémentaire, quelle serait alors la complexité dans le pire des cas de ces opérations ?

Ces opérations sont en $O(h)$, et donc, ici, pour n la taille de l'arbre, la complexité de ces opérations est en $O(2 \log(n) + 2) = O(\log(n))$.

On a bien réussi à garantir un cout logarithmique.